
Function Value
10500
10000
9500
9000
8500
8000
0
1
2
3
4
Iteration
10032
9664
9832
10656
8256
Create New Function
Modify Weights
Initialize Function
Modify Weights
Add New Component
def calculate_reward(self, vehicles_status, vehicles_status_next, action):
reward = 0
for i in range(self.num_vehicle):
# Reward based on the size of sustained platoons
sustained_platoon_size = len(vehicles_status_next[i]['platoon_partner'])
reward += sustained_platoon_size * 5 # increasing reward for platoon size
# Reward for increased vehicle density in edges for better coordination opportunities
current_edge = vehicles_status_next[i]['edge']
edge_density = len(self.edge_dict[current_edge])
reward += edge_density * 2 # encouraging higher vehicle presence in edges for forming platoons
return reward
def calculate_reward(self, vehicles_status, vehicles_status_next, action):
reward = 0
for i in range(self.num_vehicle):
# More heavily weight the reward based on the size of sustained platoons
sustained_platoon_size = len(vehicles_status_next[i]['platoon_partner'])
reward += sustained_platoon_size * 10 # significantly increasing reward for platoon size
# Slightly reduce the reward for increased vehicle density on edges
current_edge = vehicles_status_next[i]['edge']
edge_density = len(self.edge_dict[current_edge])
reward += edge_density * 1 # reducing the weight for vehicle density
return reward
def calculate_reward(self, vehicles_status, vehicles_status_next, action):
reward = 0
# Reward vehicles forming or staying in platoon
for vehicle_idx in vehicles_status_next:
platoon_size = len(vehicles_status_next[vehicle_idx]['platoon_partner'])
reward += platoon_size * self.phi_p # Encourage platoon formation
# Penalize for delay
delay_time = vehicles_status_next[vehicle_idx]['delay_time']
reward -= self.phi_s * delay_time # Penalize delays
return reward
def calculate_reward(self, vehicles_status, vehicles_status_next, action):
reward = 0
for i in range(self.num_vehicle):
# Increased reward based on the size of sustained platoons
sustained_platoon_size = len(vehicles_status_next[i]['platoon_partner'])
reward += sustained_platoon_size * 10 # significantly increasing reward for platoon size
# Slightly adjusted reward for increased vehicle density in edges for better coordination opportunities
current_edge = vehicles_status_next[i]['edge']
edge_density = len(self.edge_dict[current_edge])
reward += edge_density * 1.5 # moderately encouraging higher vehicle presence in edges for forming platoons
return reward
def calculate_reward(self, vehicles_status, vehicles_status_next, action):
reward = 0
for i in range(self.num_vehicle):
# Reward based on the size of sustained platoons
sustained_platoon_size = len(vehicles_status_next[i]['platoon_partner'])
reward += sustained_platoon_size * 5 # increasing reward for platoon size
# Reward for increased vehicle density in edges for better coordination opportunities
current_edge = vehicles_status_next[i]['edge']
edge_density = len(self.edge_dict[current_edge])
reward += edge_density * 2 # encouraging higher vehicle presence in edges for forming platoons
# Additional reward for maintaining platoon integrity on the same edge
if sustained_platoon_size > 0:
previous_platoon = vehicles_status[i]['platoon_partner']
maintained_platoon_size = len(set(previous_platoon).intersection(set(vehicles_status_next[i]['platoon_partner'])))
reward += maintained_platoon_size * 3 # reward for maintaining platoon partners
return reward